Code Reference

Understanding every part of the Zivich & Breskin
cross-fit estimator simulation, precisely

Repository: `pzivich/publications-code`, folder `DoubleCrossFit/Python`
Paper: *Machine learning for causal inference: on the use of cross-fit estimators*,
Epidemiology 2021;32(3):393–401 (arXiv:2004.10337)

July 20, 2026

How to use this document

Read Section 1 first — it explains what the whole study is trying to show and why. Then read Section 2 (where the data comes from), because every later section refers back to it. Sections 3–5 explain the machinery one piece at a time. Section 6 explains the six scripts you actually run, and Section 7 explains exactly what the numbers they print mean.
Every formula here was read directly out of the code, not reconstructed from the paper. Where the code does something the paper does not describe, this is flagged explicitly.

Contents

1 The big picture: what this study is asking

1.1 The scientific question

We want to estimate a **causal effect**: how much would the risk of an outcome Y change if everyone took a drug ($A = 1$) compared with if nobody took it ($A = 0$)? The quantity of interest is the **average treatment effect** on the risk-difference scale:

$$\psi \;=\; \mathbb{E}[Y^{a=1}] - \mathbb{E}[Y^{a=0}],$$

where Y^a is the potential outcome under treatment a. In this simulation the true value is known exactly by construction:

$$\boxed{\psi_{\text{true}} = -0.1081508}$$

This number appears hard-coded as `truth` at the top of every simulation script.

Because treatment is not randomised, we must adjust for confounders X. Doing so requires estimating *nuisance functions* — quantities we do not care about for their own sake, but which we need in order to get at ψ:

- the **propensity score** $g(X) = \Pr(A = 1 \mid X)$, and
- the **outcome model** $Q(A, X) = \mathbb{E}[Y \mid A, X]$.

1.2 The methodological question

Modern practice is to estimate those nuisance functions with flexible machine learning (here, a *super learner*). The theoretical worry is that machine learning converges to the truth *slowly*, and that this slowness contaminates the final effect estimate, producing bias and confidence intervals that are too narrow. This is often called **plug-in bias** or the *slow convergence* problem.

Theory says two things fix it:

1. use an estimator built on the **efficient influence function** — which is what AIPW and TMLE are — rather than a naive plug-in like g-computation;
2. use **cross-fitting** (sample splitting), so that the data used to fit the nuisance models is not the same data used to evaluate the effect.

The paper's job is to *demonstrate* this empirically. Hence the design: six estimators × three ways of estimating the nuisance functions.

1.3 The 6 × 3 design

Estimator	Script	Uses which nuisance functions?
G-computation	`sim_gform.py`	outcome model Q only
IPW	`sim_iptw.py`	propensity score g only
AIPW	`sim_aipw.py`	both g and Q (doubly robust)
TMLE	`sim_tmle.py`	both g and Q (doubly robust)
Double cross-fit AIPW	`sim_dcaipw.py`	both, with sample splitting
Double cross-fit TMLE	`sim_dctmle.py`	both, with sample splitting

Each script is run three times, controlled by one line near the top:

```
setup = 1  # options include: 1-correct parametric model, 2-main-terms model, 3-
    machine learning
```

`setup`	What the nuisance models are	What we should see
1	Correctly specified parametric models	Everything works: no bias, 95% coverage
2	Main-terms (deliberately wrong) models	Bias; coverage below 95%
3	Super learner (machine learning)	**The interesting case**

The headline result you are trying to reproduce

Under `setup = 3` (machine learning), **g-computation** should show noticeable bias and confidence intervals that cover the truth much less than 95% of the time. The **doubly robust** estimators (AIPW, TMLE) — and especially the **double cross-fit** versions — should be much closer to unbiased with coverage near 95%. That contrast is the entire point of the paper.

2 `dgm.py` — where the data comes from

This script is run **once**. It creates every dataset the study will ever use and writes them into a single file, `statin_sim_data.csv` (about 346 MB).

2.1 Size and reproducibility

```
n = 3000          # size of each sample
sims = 2000       # number of repeated samples
np.random.seed(1015033030)
```

So the file contains $2000 \times 3000 = 6{,}000{,}000$ rows: 2,000 independent simulated studies, each with 3,000 people. A column `sim_id` (running $1 \ldots 2000$) says which study each row belongs to. The fixed seed means anyone re-running `dgm.py` gets byte-identical data — this is why reproduction is possible at all.

Note for your supervisor

The sample size really is $n = 3000$, with 2,000 simulated datasets. This is worth confirming because it was misremembered as 500 in conversation.

2.2 The scenario

The setting is a *statin / cardiovascular disease* example. Variables:

Column	Meaning	Role
`age`	age in years	confounder X
`ldl_log`	log LDL cholesterol	confounder X
`diabetes`	diabetes indicator	confounder X
`risk_score`	continuous risk score	confounder X
`risk_score_cat`	risk score in 4 categories	confounder X
`statin`	took a statin (0/1)	treatment A
`Y`	outcome (0/1)	outcome Y
`sim_id`	which of the 2000 studies	bookkeeping

2.3 Step 1 — the confounders

```
df['age'] = np.round(trapezoidal(40, 40, 60, 75, size=n * sims), 0)
df['ldl_log'] = 0.005 * df['age'] + np.random.normal(np.log(100), 0.18, size=n *
    sims)
df['diabetes'] = np.random.binomial(n=1, p=logistic.cdf(-4.23 + 0.03 * df['
    ldl_log']
                - 0.02 * df['age'] + 0.0009 * df['age'] ** 2), size=n * sims)
```

Age follows a *trapezoidal* distribution between 40 and 75 (flat between 40 and 60, then declining) — drawn using `zepid`'s `trapezoidal` function. LDL depends on age. Diabetes depends on both age and LDL. So the confounders are already related to each other, as in real data.

A latent variable `frailty` is also generated and used inside `risk_score`, but — importantly — **`frailty` is never saved to the CSV**. Its influence is tiny (coefficient 0.01), and because `risk_score` is saved, no unmeasured confounding of consequence is introduced.

The `risk_score` is a deliberately awkward nonlinear function:

```
df['risk_score'] = logistic.cdf(4.299 + 3.501*df['diabetes'] - 2.07*df['age_ln']
                  + 0.051*df['age_ln']**2 + 4.090*df['ldl_log']
                  - 1.04*df['age_ln']*df['ldl_log'] + 0.01*df['frailty'])
```

Note the squared term and the *interaction* `age_ln * ldl_log`. This nonlinearity is exactly why a naive "main terms only" model (setup 2) is wrong, and why flexible machine learning (setup 3) is attractive in the first place.

`risk_score_cat` bins the score at cut-points 0.05, 0.075, 0.2 into categories $0, 1, 2, 3$.

2.4 Step 2 — who gets treated (the propensity score)

```
df['statin_pr'] = logistic.cdf(-3.471 + 1.390*df['diabetes'] + 0.112*df['ldl_log
    ']
                   + 0.973*np.where(df['ldl_log'] > np.log(160), 1, 0)
                   - 0.046*(df['age'] - 30) + 0.003*(df['age'] - 30)**2
                   + 0.273*(risk_score_cat==1) + 1.592*(risk_score_cat==2)
                   + 2.641*(risk_score_cat==3))
df['statin'] = np.random.binomial(n=1, p=df['statin_pr'], size=n*sims)
```

This is the **true propensity score**. Read the terms carefully, because they tell you exactly what a "correctly specified" propensity model must contain:

- `diabetes`, `ldl_log` — plain linear terms;
- a **threshold** indicator for LDL above log(160);
- `age` centred at 30, *and its square*;
- the risk score as a **categorical** variable, not a linear one.

Why `setup = 1` looks the way it does

Compare the above with the setup-1 propensity model in the scripts:

```
g_model = 'diabetes + ldl_log + ldl_160 + age_30 + age_30_sq + C(
    risk_score_cat)'
```

It matches the data-generating formula term for term — that is precisely what "correctly specified" means here. The setup-2 model (`'diabetes + age + risk_score + ldl_log'`) omits the threshold, the square, and the categorisation, which is why it is biased.

2.5 Step 3 — the potential outcomes

Both potential outcomes are generated for every person:

```
prY1 = logistic.cdf(-6.25
       - 0.75                                                    # treatment
           effect
       + 0.35*np.where(df['ldl_log'] < np.log(130), 5-df['ldl_log'], 0)
       + 0.45*(np.sqrt(df['age']-39)) + 1.75*df['diabetes']
       + 0.29*(np.exp(df['risk_score']+1))
       + 0.14*np.where(df['ldl_log'] > np.log(120), df['ldl_log']**2, 0))
df['Y1'] = np.random.binomial(n=1, p=prY1, size=n*sims)
```

`prY0` is the identical expression *minus* the two treatment terms (-0.75 and the $0.35\times\ldots$ spline-like term).

Two things to notice. First, the treatment effect is **not constant**: the term `0.35 * (5 - ldl_log)` for people with LDL below log(130) means the benefit of statins depends on LDL — an *effect modification*. That is why the correct outcome model contains an interaction `statin:ldl_130`. Second, the outcome model involves $\sqrt{\text{age} - 39}$, $e^{\text{risk\_score}+1}$ and a truncated quadratic in LDL — all strongly nonlinear.

Finally, causal consistency is imposed:

```
df['Y'] = np.where(df['statin'] == 1, df['Y1'], df['Y0'])    # causal consistency
```

Each person's observed outcome is the potential outcome matching the treatment they actually received. **`Y1` and `Y0` are then dropped** — only `Y` is saved, as in real life.

Where the number -0.1081508 comes from

The authors left the calculation in the file as a comment:

```
# print(np.mean(df['Y1'] - df['Y0']))
# truth = -0.1081508
```

Because both potential outcomes are known for all 6 million rows, the true average treatment effect can be computed directly: statins reduce risk by about 10.8 percentage points. Every simulation script compares its estimate against this number.

2.6 What is saved

```
columns_to_keep = ['sim_id', 'Y', 'statin', 'age', 'ldl_log', 'diabetes',
                   'risk_score', 'risk_score_cat']
df[columns_to_keep].to_csv("statin_sim_data.csv", index=False)
```

The unobservable quantities (`Y1`, `Y0`, `statin_pr`, `frailty`) are deliberately discarded, so the estimators face the same information a real analyst would.

3 `super_learner.py` — the machine learning engine

This file supplies the estimator used whenever `setup = 3`. It is a self-contained implementation of the *super learner* (also called *stacking*), adapted from `alexpkeil1/SuPyLearner`.

3.1 The idea in one paragraph

You have several candidate prediction algorithms and no idea which is best. The super learner runs K-fold cross-validation to see how each one performs on data it did not see, then forms a **weighted average** of them, choosing the weights that minimise cross-validated loss. The result is guaranteed (asymptotically) to do at least as well as the best single candidate.

3.2 The library of candidate algorithms

```
log_b   = LogisticRegression(penalty='none', solver='lbfgs', max_iter=1000)
rdf_b   = RandomForestClassifier(n_estimators=500, min_samples_leaf=20)
gam1_b  = LogisticGAM(n_splines=4, lam=0.6)
gam2_b  = LogisticGAM(n_splines=6, lam=0.6)
nn1_b   = MLPClassifier(hidden_layer_sizes=(4,), activation='relu',
                        solver='lbfgs', max_iter=2000)
emp_b   = EmpiricalMean()
lib      = [log_b, gam1_b, gam2_b, rdf_b, nn1_b, emp_b]
libnames = ["Logit", "GAM1", "GAM2", "Random Forest", "Neural-Net", "Mean"]
```

Six candidates, for binary variables:

Name	What it is	Character
Logit	unpenalised logistic regression	rigid, parametric
GAM1	generalised additive model, 4 splines	smooth, flexible
GAM2	generalised additive model, 6 splines	smoother, more flexible
Random Forest	500 trees, min. 20 obs. per leaf	very flexible, non-smooth
Neural-Net	MLP, one hidden layer of 4 units	very flexible
Mean	predicts the sample mean, ignores X	maximally rigid baseline

This is the neural net your supervisor asked about

`nn1_b` — `MLPClassifier(hidden_layer_sizes=(4,))` — is a small neural network with a single hidden layer of four neurons. It is one of the six candidates, and it is the specific component your supervisor said he would "never dream of using". Removing it later means deleting `nn1_b` from `lib` and `"Neural-Net"` from `libnames` (and `nn1_c` in the continuous block). **Do not change it during the reproduction.**

A parallel `'continuous'` library exists (`LinearRegression`, `RandomForestRegressor`, two identity-link `GAM`s, `MLPRegressor`, `EmpiricalMean`) but this simulation only ever uses the `'binary'` branch, because both A and Y are binary.

3.3 The K parameter — and why it dominates your runtime

```
def superlearnersetup(var_type, K=5):
    ...
    sl = SuperLearner(lib, libnames, loss="nloglik", K=K, print_results=False)
```

The production scripts call it with `K=10`:

```
q_estimator = superlearnersetup(var_type='binary', K=10)
```

whereas the single-dataset example `sim_single_example.py` uses `K=5`.

Fitting cost — the origin of the runtime problem

One `fit()` of the super learner trains each of the 6 candidates K times for cross-validation, *plus* once more on the full data:

$$\text{model fits per super-learner fit} \;=\; 6 \times (K+1) \;=\; 66 \text{ when } K = 10.$$

Since a random forest of 500 trees and a neural network are among those, a single super learner fit on $n = 3000$ takes roughly 25–50 seconds. Estimators that refit the super learner hundreds of times *per dataset* therefore become astronomically expensive — see Section 5.5.

3.4 How `fit()` works, step by step

```
n = len(y)
folds = ms.KFold(self.K)
y_pred_cv = np.empty(shape=(n, self.n_estimators))
for train_index, test_index in folds.split(range(n)):
    for aa in range(self.n_estimators):
        est = clone(self.library[aa])
        est.fit(X_train, y_train)
        y_pred_cv[test_index, aa] = self._get_pred(est, X_test)
self.coef = self._get_coefs(y, y_pred_cv)
```

1. **Cross-validated predictions.** Split the rows into K folds. For each fold, train all six candidates on the other $K-1$ folds and predict the held-out fold. This fills an $n \times 6$ matrix `y_pred_cv` of *honest* (out-of-sample) predictions.
2. **Find the weights.** Choose weights α minimising the loss between y and the weighted combination of those columns.
3. **Refit on everything.** Every candidate is then refit on the *full* data and stored in `fitted_library`; the stored `coef` weights are reused.

Subtlety: `KFold` is not shuffled

`ms.KFold(self.K)` is created **without** `shuffle=True`, so the folds are contiguous blocks of rows in their existing order. Because rows within a simulated dataset are exchangeable this is harmless here, but it is a genuine trap when data arrives sorted.

3.5 The loss function and the weights

The binary library is built with `loss="nloglik"` (negative log-likelihood). With that loss, the combination is formed *on the logit scale*:

```
comb = _inv_logit(np.dot(_logit(_trim(y_pred_mat, self.bound)), coef))
```

i.e.

$$\hat{p}_{\mathrm{SL}}(x) \;=\; \operatorname{expit}\Big(\textstyle\sum_j \alpha_j \;\operatorname{logit}\big(\hat{p}_j(x)\big)\Big),$$

with predictions first trimmed into $[10^{-5},\, 1-10^{-5}]$ so the logit is finite. The risk being minimised is

$$\mathcal{R}(\alpha) \;=\; -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \log \hat{p}_{\mathrm{SL}}(x_i) + (1-y_i)\log\big(1-\hat{p}_{\mathrm{SL}}(x_i)\big)\Big].$$

Weights are found by `SLSQP` constrained optimisation with $\alpha_j \in [0,1]$ and $\sum_j \alpha_j = 1$:

```
def constr(x): return np.array([np.sum(x) - 1])
coef_init, b, c, d, e = fmin_slsqp(ff, x0, f_eqcons=constr, bounds=bds, ...)
coef_init[coef_init < np.sqrt(np.finfo(np.double).eps)] = 0
coef = coef_init / np.sum(coef_init)
```

Negligible weights are snapped to exactly zero and the rest renormalised to sum to one. So the super learner output is a genuine *convex combination* of the six candidates.

Two deprecated-Python idioms live here

`_get_coefs` uses `if self.coef_method is 'SLSQP'` — identity comparison on strings, not equality. This worked on older CPython because of string interning; on modern Python it emits `SyntaxWarning` and is fragile. Similarly `if c['warnflag'] is not 0`. This is one reason the code is run on a pinned, era-appropriate Python environment rather than being edited.

3.6 `EmpiricalMean` — the deliberately stupid candidate

```
class EmpiricalMean(BaseEstimator):
    def fit(self, X, y):        self.empirical_mean = np.mean(y)
    def predict(self, X):       return np.array([self.empirical_mean] * X.shape
        [0])
```

It ignores the covariates entirely. Including it is good practice: if no candidate can beat the overall mean, the super learner will fall back to it rather than overfit.

4 `estimators.py` — the six estimators

This is the biggest file (about 1,370 lines). You never run it directly; the simulation scripts import from it. It contains six estimator classes and a handful of private helper functions.

4.1 The shared pattern

Every class follows the same three-step usage:

```
est = SomeEstimator(dataframe, treatment='statin', outcome='Y')
est.treatment_model(g_model, g_estimator, bound=0.01)    # propensity score  (not
    for GFormula)
est.outcome_model(q_model, q_estimator)                  # outcome model     (not
    for IPTW)
est.fit()
est.risk_difference                                      # the answer
```

Two conventions used throughout, worth learning once:

- **Design matrices are built by `patsy`, with the intercept removed.** Every class does

  ```
  xdata = np.asarray(patsy.dmatrix(covariates + ' - 1', self.df))
  ```

  The appended `' - 1'` strips patsy's intercept column, because the scikit-learn estimator supplies its own intercept.

- **Nuisance models are fitted by duck typing.** The code calls `estimator.fit(X=..., y=...)` and then prefers `predict_proba(...)[:, 1]`, falling back to `predict(...)`. This is why a scikit-learn `LogisticRegression` and the custom `SuperLearner` are interchangeable — both expose that interface.

4.2 Counterfactual prediction: the trick used everywhere

To ask "what if everyone were treated?", every outcome-model-based estimator does the same thing: copy the data, overwrite the treatment column, and rebuild the design matrix.

```
df = self.df.copy()
df[self.treatment] = 1
x_all = np.asarray(patsy.dmatrix(self._y_covariates + ' - 1', df))
r_all = fm.predict_proba(x_all)[:, 1]
```

Why the design matrix must be rebuilt, not reused

If the formula contains an interaction such as `statin:ldl_130`, then changing `statin` must also change that interaction column. Rebuilding the matrix from the modified data handles this automatically. This is also why the outcome formula **must** contain the treatment term — if it does not, the two counterfactual predictions are identical and the estimated effect collapses to zero, *with no error raised.*

4.3 `GFormula` — g-computation

What it computes

Fit $\hat{Q}(A, X) = \hat{\mathbb{E}}[Y \mid A, X]$ once on the observed data. Then predict every person's outcome twice — once pretending they were treated, once pretending they were not — and average:

$$\hat{\psi}_{\text{g-comp}} = \underbrace{\frac{1}{n}\sum_{i=1}^{n} \hat{Q}(1, X_i)}_{\texttt{risk\_all}} - \underbrace{\frac{1}{n}\sum_{i=1}^{n} \hat{Q}(0, X_i)}_{\texttt{risk\_none}}$$

```
self.risk_all = np.mean(r_all)
self.risk_none = np.mean(r_non)
self.risk_difference = self.risk_all - self.risk_none
```

The critical fact: no standard error

`GFormula` produces *no* confidence interval

The class computes `risk_difference` and nothing else. Its own docstring says: *"Confidence intervals are not calculated. A bootstrapping procedure should be used."*
This single fact explains the entire runtime problem. Because the variance must come from outside the class, `sim_gform.py` wraps the whole estimator in a bootstrap loop of `bs_samples = 250` resamples. So g-computation needs **251 model fits per dataset**, while AIPW and TMLE need only two. When the model being fitted is a super learner, this is the difference between hours and months.

Things to know

- `__init__` does `df.copy().dropna().reset_index()`: it drops any row with a missing value *anywhere*, and (because `drop=True` is not passed) adds a leftover column called `index`.
- There is no probability bounding and no `bound` argument.
- There is no continuous/binary outcome detection; the `predict_proba` versus `predict` branch handles it implicitly.

4.4 `IPTW` — inverse probability weighting

What it computes

Fit the propensity score $\hat{g}(X) = \hat{\Pr}(A = 1 \mid X)$, then weight each person by the inverse of the probability of the treatment they actually received:

$$w_i = \frac{A_i}{\hat{g}(X_i)} + \frac{1 - A_i}{1 - \hat{g}(X_i)}$$

```
self.df['_iptw_'] = (self.df[self.treatment] / pr_a) + ((1 - self.df[self.
    treatment]) / (1 - pr_a))
```

These are *unstabilised* weights. Intuitively, people who received an unlikely treatment stand in for the many similar people who did not.

The effect is then read off a weighted regression of Y on A — fitted with an **identity link**, so that the coefficient is a risk *difference* rather than a log-odds:

```
ind = sm.cov_struct.Independence()
f = sm.families.family.Binomial(sm.families.links.identity)
fm = smf.gee('Y ~ statin', df.index, df, cov_struct=ind, family=f, weights=df['
    _iptw_']).fit()
self.risk_difference = np.asarray(fm.params)[1]
self.risk_difference_se = np.asarray(fm.bse)[1]
```

Index `[1]` is the coefficient on `statin` (index `[0]` is the intercept). The standard error is the GEE **robust (sandwich)** standard error. Because `df.index` is passed as the grouping variable and every row has a unique index, each observation is its own cluster — so this is the ordinary robust variance.

Two things that will surprise you in `IPTW.fit()`

1. The regression formula is hard-coded as `'Y ~ statin'`. The `treatment` and `outcome` names you passed to the constructor are ignored at this step. The class only works on data whose columns are literally named `Y` and `statin`. It is fine here — that is what the simulation data uses — but it is not a general-purpose class.

2. The standard error is deliberately conservative. It treats the estimated propensity score as if it were known in advance, ignoring the uncertainty from having estimated it. The docstring acknowledges this. Expect IPTW's average standard error to run a little *larger* than its empirical standard error, and coverage slightly above 95%.

Bounding

`treatment_model(covariates, estimator, bound=False)` — the default is *no* bounding, but every simulation script passes `bound=0.01`, which truncates the propensity score into [0.01, 0.99] via `_bounding_`. This prevents division by values near zero. Note that `_bounding_` rejects integers: you must write `0.01`, not `0`.

4.5 `AIPTW` — augmented IPW (doubly robust)

What it computes

AIPW combines both nuisance models so that the estimate is consistent if *either* one is correct — the **double robustness** property. For each person the code builds two pseudo-outcomes:

```
y1 = np.where(self.df[self.treatment] == 1,
              (y_obs / ps_g1) - ((py_a1 * ps_g0) / ps_g1),
              py_a1)
y0 = np.where(self.df[self.treatment] == 1,
              py_a0,
              (y_obs / ps_g0 - ((py_a0 * ps_g1) / ps_g0)))
```

which is the standard augmented estimator written out branch by branch:

$$\hat{Y}_i^1 = \frac{A_i Y_i}{\hat{g}(X_i)} + \left(1 - \frac{A_i}{\hat{g}(X_i)}\right)\hat{Q}(1, X_i), \qquad \hat{Y}_i^0 = \frac{(1-A_i)Y_i}{1-\hat{g}(X_i)} + \left(1 - \frac{1-A_i}{1-\hat{g}(X_i)}\right)\hat{Q}(0, X_i)$$

$$\hat{\psi}_{\text{AIPW}} = \frac{1}{n}\sum_{i=1}^{n}\left(\hat{Y}_i^1 - \hat{Y}_i^0\right)$$

Read it as: *start from the outcome-model prediction, then correct it using the weighted residual.* If the outcome model is perfect the correction term has mean zero; if the propensity model is perfect the weighting alone is unbiased. Either way you win.

Variance — the influence function

```
self.risk_difference_se = np.sqrt(np.var((y1 - y0) - self.risk_difference, ddof
    =1) / self.df.shape[0])
self.risk_difference_ci = [self.risk_difference - 1.96 * self.risk_difference_se
    ,
                           self.risk_difference + 1.96 * self.risk_difference_se
                              ]
```

The **influence function** is $D_i = (\hat{Y}_i^1 - \hat{Y}_i^0) - \hat{\psi}$, and

$$\widehat{\text{Var}}(\hat{\psi}) = \frac{1}{n}\,\widehat{\text{Var}}(D_i).$$

This is the theoretically efficient variance — but it treats the nuisance fits as known, which is exactly the assumption that fails when they are estimated by machine learning without cross-fitting.

A harmless oddity you will notice

`np.var((y1 - y0) - self.risk_difference, ddof=1)` subtracts a constant before taking a variance. Since `np.var` already centres on the sample mean, the subtraction changes nothing. It is stylistic, not a correction — do not read meaning into it.

4.6 `TMLE` — targeted maximum likelihood

TMLE reaches the same theoretical place as AIPW by a different route: instead of adding a correction to the *estimate*, it nudges the outcome *predictions* themselves and then plugs them in.

Step 1 — initial predictions

Fit the outcome model, giving $\hat{Q}(1, X)$, $\hat{Q}(0, X)$ (`QA1W`, `QA0W`), and $\hat{Q}(A, X)$ at the observed treatment (`QAW`). Fit the propensity model, giving `g1W` and `g0W = 1 - g1W`.

Step 2 — the clever covariate

```
H1W = self.df[self.exposure] / self.g1W
H0W = -(1 - self.df[self.exposure]) / self.g0W
HAW = H1W + H0W
```

$$H(A, X) = \frac{A}{\hat{g}(X)} - \frac{1 - A}{1 - \hat{g}(X)}$$

Note the **minus sign is built into `H0W`**. This matters in step 4.

Step 3 — the targeting (fluctuation) step

```
f = sm.families.family.Binomial()
log = sm.GLM(y, np.column_stack((H1W, H0W)),
             offset=np.log(self.probability_to_odds(self.QAW)),
             family=f, missing='drop').fit()
self._epsilon = log.params
```

A logistic regression of Y on the two clever covariates, **with no intercept**, and with an **offset equal to the logit of the initial prediction**:

$$\operatorname{logit} \hat{Q}^*(A, X) = \operatorname{logit} \hat{Q}(A, X) + \epsilon_1 H_1(A, X) + \epsilon_2 H_0(A, X)$$

Since `probability_to_odds(p) = p/(1-p)`, the expression `np.log(probability_to_odds(q))` *is* the logit — it is simply never named that in the code. Fitting this tiny two-parameter model is the "targeting": it moves the predictions just enough to solve the efficient influence function equation.

Step 4 — updated predictions and the estimate

```
Qstar1 = logistic.cdf(np.log(self.probability_to_odds(self.QA1W)) + self.
    _epsilon[0] / self.g1W)
Qstar0 = logistic.cdf(np.log(self.probability_to_odds(self.QA0W)) - self.
    _epsilon[1] / self.g0W)
self.risk_difference = np.nanmean(Qstar1 - Qstar0)
```

Setting $A = 1$ makes $H_1 = 1/\hat{g}$ and $H_0 = 0$, hence the $+\epsilon_1/g_1$; setting $A = 0$ makes $H_0 = -1/(1-\hat{g})$, hence the $-\epsilon_2/g_0$ — that is where the minus sign from step 2 reappears. `logistic.cdf` is the inverse logit.

Variance

```
ic = HAW * (self.df[self.outcome] - Qstar) + (Qstar1 - Qstar0) - self.
    risk_difference
varIC = np.nanvar(ic, ddof=1) / self.df.shape[0]
```

$$D_i \;=\; H(A_i, X_i)\big(Y_i - \hat{Q}^*(A_i, X_i)\big) \;+\; \big(\hat{Q}^*(1, X_i) - \hat{Q}^*(0, X_i)\big) \;-\; \hat{\psi}$$

the textbook efficient influence curve, with $\widehat{\mathrm{Var}}(\hat{\psi}) = \widehat{\mathrm{Var}}(D)/n$ and a hard-coded $z = 1.96$.

Traps in `TMLE`

- The treatment attribute is called `self.exposure` here, not `self.treatment` as in every other class.
- `alpha=0.05` is accepted by the constructor and then **never used** — the critical value is hard-coded to 1.96. Changing `alpha` does nothing.
- For *continuous* outcomes the outcome column is destructively rescaled to $(0, 1)$ inside `__init__` and unscaled again in `fit()`. Irrelevant here (our Y is binary) but startling if you inspect `tmle.df`.
- `summary()` has a copy-paste bug: it checks the exposure model twice and never checks the outcome model.

5 The double cross-fit estimators — the heart of the paper

`DoubleCrossfitAIPTW` and `DoubleCrossfitTMLE` are structurally identical; they differ only in the final estimating equation (`_aipw_calculator` versus `_tmle_calculator`). Understanding one gives you both.

5.1 Why cross-fitting exists

In `AIPTW` and `TMLE` above, the nuisance models are fitted on the data and then used to predict *that same data*. With a rigid parametric model this is fine. With a flexible learner it is not: the model partly memorises the data, its predictions are optimistically good, and the resulting bias does not vanish at the rate the theory requires. **Cross-fitting removes this by ensuring that no observation's estimate ever uses a model that saw that observation.**

5.2 The three-way split

```
def _sample_split_(data, seed):
    """Background function to split data into three non-overlapping pieces"""
    n = int(data.shape[0] / 3)
    s1 = data.sample(n=n, random_state=seed)
    s2 = data.loc[data.index.difference(s1.index)].sample(n=n, random_state=seed
        )
    s3 = data.loc[data.index.difference(s1.index) & data.index.difference(s2.
        index)]
    return s1, s2, s3
```

The data is cut into **exactly three** non-overlapping parts. The number three is hard-coded (the file even carries a `# TODO generalize this to arbitrary K`). Splits 1 and 2 get exactly $\lfloor n/3 \rfloor$ rows; split 3 absorbs the remainder, so it may be one or two rows larger.

5.3 The rotation — what makes it *double* cross-fitting

Both nuisance models are fitted separately on *each* of the three splits, giving six fitted models. Then:

```
s1_predictions = self._generate_predictions(sample_split[0], a_model_v=a_models
    [1], y_model_v=y_models[2])
s2_predictions = self._generate_predictions(sample_split[1], a_model_v=a_models
    [2], y_model_v=y_models[0])
s3_predictions = self._generate_predictions(sample_split[2], a_model_v=a_models
    [0], y_model_v=y_models[1])
```

Estimate evaluated on	Propensity model trained on	Outcome model trained on
split 1	split 2	split 3
split 2	split 3	split 1
split 3	split 1	split 2

The key idea, in one sentence

For every row of data, the propensity score comes from a model that never saw it, the outcome prediction comes from a *different* model that also never saw it, **and those two models were themselves trained on two disjoint sets of people**. That last requirement — two *separate* independent samples for the two nuisance functions — is what the word "*double*" refers to.

Each nuisance model therefore trains on only $n/3$ observations, not the $2n/3$ you might expect from ordinary K-fold cross-fitting. Every observation still receives exactly one prediction, so the final average uses the full sample.

5.4 Combining across many random splits

A single three-way split is arbitrary — a different random split gives a different answer. So the whole procedure is repeated:

```
for j in range(resamples):
    split_samples = _sample_split_(self.df, seed=None)
    result = self._single_crossfit_(sample_split=split_samples)
    rd_point.append(result[0])
    rd_var.append(result[1])
```

with `resamples = 100` in the simulation scripts (`n_diff_splits = 100`). Note `seed=None`: each repetition genuinely re-randomises the partition.

The 100 answers are then pooled:

```
if method == 'median':
    self.risk_difference = np.median(rd_point)
    self.risk_difference_se = np.sqrt(np.median(rd_var + (rd_point - self.
        risk_difference)**2))
```

$$\hat{\psi} = \text{median}_j\,(\hat{\psi}_j), \qquad \widehat{\text{SE}} = \sqrt{\text{median}_j\left(\hat{V}_j + (\hat{\psi}_j - \hat{\psi})^2\right)}$$

Read that standard-error formula carefully

It is **not** "median of the variances plus a between-split term". The correction $(\hat{\psi}_j - \hat{\psi})^2$ is added *inside* the median, per split. Because the median is not additive, these are genuinely different quantities. This is the median-adjusted variance of Chernozhukov et al.: it inflates the standard error to account for the fact that the split itself was random. (With `method='mean'` the two readings do coincide.)

5.5 Where all the computing time goes

Per dataset, one double cross-fit run costs:

$$\underbrace{100}_{\text{resamples}} \times \underbrace{3}_{\text{splits}} \times \underbrace{2}_{\text{nuisance models}} = 600 \text{ model fits.}$$

Under `setup = 3` each of those fits is a super learner, and each super learner is itself $6\times(10+1) = 66$ underlying model fits. So a *single dataset* costs about $600 \times 66 \approx 40{,}000$ model fits — and the study asks for 2,000 datasets.

Script (setup 3)	Super-learner fits per dataset	Feasible on a laptop?
`sim_iptw.py`	1	yes (about a day)
`sim_aipw.py`	2	yes
`sim_tmle.py`	2	yes
`sim_gform.py`	251 (bootstrap)	**no**
`sim_dcaipw.py`	600	**no**
`sim_dctmle.py`	600	**no**

The runtime argument in one line

IPTW at setup 3 does *one* super-learner fit per dataset and takes roughly a day. The double cross-fit scripts do *six hundred*. That is the whole explanation for why these three runs need a computing cluster — and it is arithmetic on the authors' own settings, not an estimate.

5.6 Gotchas in the double cross-fit classes

- **`random_state` does nothing.** The constructor accepts it and stores it in `self._seed_`, which is never read again; `fit()` always calls `_sample_split_(self.df, seed=None)`. Reproducibility comes only from the global NumPy seed. (Arguably correct — honouring it would make all 100 resamples identical — but the argument is misleading.)
- **Only the propensity model is bounded**, and only if `bound` was passed. Outcome predictions are never bounded.
- **The estimate is pooled across all rows**, not averaged over the three folds, so the slightly larger third split carries slightly more weight.
- **Failures are swallowed.** The simulation scripts wrap each dataset in `try/except` and record `np.nan` on any error, so a run can complete with missing rows and no visible complaint — always check for NaNs in the output CSV.

6 The six simulation scripts

These are the files you actually run. All six share an identical skeleton, so learn it once.

6.1 The common anatomy

```
# 1. PARAMETERS
setup = 1
file_path_to_save_results = "gform_results" + str(setup) + ".csv"

# 2. DATA
df = pd.read_csv("statin_sim_data.csv")
truth = -0.1081508
samples = list(df['sim_id'].unique())          # [1, 2, ..., 2000]

# 3. MODEL SPECIFICATION  (the if/elif/elif on setup)
if setup == 1:   ...correct models...
elif setup == 2: ...main-terms models...
elif setup == 3: ...super learner...
else: raise ValueError("Invalid setup choice")

# 4. THE LOOP over 2000 datasets
for i in samples:
    dfs = df.loc[df['sim_id'] == i].copy()
    ...estimate...
    bias.append(estimate - truth)

# 5. SUMMARISE and SAVE
results.to_csv(file_path_to_save_results, index=False)
```

Note in step 2 that **the entire 346 MB file is loaded into memory once**, then step 4 slices out one simulated study at a time with `df.loc[df['sim_id'] == i]`. Each iteration is a completely independent analysis of 3,000 people — which is precisely why the work could be split across cluster nodes.

The output file name is built from the `setup` variable automatically, so `setup = 2` writes `gform_results2.csv`. This is why changing that one digit is sufficient to produce a different run — and why the copies used in this project (`run_sim_*_setup2.py`) differ from the originals by exactly one character.

6.2 The three model specifications

Setup 1 — correctly specified

```
df['ldl_160']   = np.where(df['ldl_log'] > np.log(160), 1, 0)
df['age_30']    = df['age'] - 30
df['age_30_sq'] = (df['age'] - 30)**2
df['ldl_130']   = np.where(df['ldl_log'] < np.log(130), 5-df['ldl_log'], 0)
df['age_sqrt']  = np.sqrt(df['age']-39)
df['risk_exp']  = np.exp(df['risk_score']+1)
df['ldl_120']   = np.where(df['ldl_log'] > np.log(120), df['ldl_log']**2, 0)

g_model = 'diabetes + ldl_log + ldl_160 + age_30 + age_30_sq + C(risk_score_cat)
    '
q_model = 'statin + statin:ldl_130 + age_sqrt + diabetes + risk_exp + ldl_120'
```

Compare these term by term with `dgm.py` (Section 2) and you will find they match the data-generating equations exactly. Every derived variable here exists to reproduce one term of the truth:

Derived variable	Reproduces	Used in
`ldl_160`	LDL threshold indicator in the treatment model	g
`age_30`, `age_30_sq`	centred age and its square	g
`ldl_130`	the effect-modification term	Q
`age_sqrt`	$\sqrt{\text{age} - 39}$	Q
`risk_exp`	$e^{\text{risk\_score}+1}$	Q
`ldl_120`	truncated quadratic in LDL	Q

The term `statin:ldl_130` is the interaction encoding that the treatment effect depends on LDL. `C(risk_score_cat)` tells patsy to treat the risk category as a factor (dummy variables), matching how it entered the true propensity score.

Setup 2 — main terms only (deliberately wrong)

```
g_model = 'diabetes + age + risk_score + ldl_log'
q_model = 'statin + diabetes + age + risk_score + ldl_log'
```

Every nonlinearity, threshold, square, categorisation and interaction is gone. This is what a hurried analyst would write. It is *misspecified*, and the results should show it.

Setup 3 — machine learning

```
g_model = 'diabetes + age + risk_score + ldl_log'
q_model = 'statin + diabetes + age + risk_score + ldl_log'
g_estimator = superlearnersetup(var_type='binary', K=10)
q_estimator = superlearnersetup(var_type='binary', K=10)
```

The *formulas* are the same simple main-terms strings as setup 2 — but the *estimator* is now the super learner, which can discover nonlinearity by itself. This is the honest modern workflow, and the case the paper is really about.

Why setup 3 is the interesting one

Setup 1 shows the methods work when you already know the answer. Setup 2 shows they break when your model is wrong. Setup 3 asks the real question: *if we let machine learning find the shape of the nuisance functions, does the causal estimate still behave?* The paper's answer is that it depends entirely on which estimator you use — and that is the contrast you are reproducing.

6.3 Script-by-script differences

`sim_gform.py` — the only one with a bootstrap

```
bs_samples = 250

for i in samples:
    dfs = df.loc[df['sim_id'] == i].copy()
    gform = GFormula(dfs, treatment='statin', outcome='Y')
    gform.outcome_model(covariates=q_model, estimator=q_estimator)
    gform.fit()
    rd = gform.risk_difference
    bias.append(rd - truth)

    bse = []
    for j in range(bs_samples):                    # <-- 250 more fits
        dfsr = dfs.sample(n=dfs.shape[0], replace=True)
```

```
        gform = GFormula(dfsr, treatment='statin', outcome='Y')
        gform.outcome_model(covariates=q_model, estimator=q_estimator)
        gform.fit()
        bse.append(gform.risk_difference)

    stderr.append(np.std(bse, ddof=1))
    lcl.append(rd - 1.96*np.std(bse, ddof=1))
    ucl.append(rd + 1.96*np.std(bse, ddof=1))
```

The inner loop resamples the 3,000 people *with replacement* 250 times and refits. The standard deviation of those 250 estimates is the bootstrap standard error. This exists only because `GFormula` provides no analytic variance.

`sim_iptw.py`, `sim_aipw.py`, `sim_tmle.py` — the simple ones

No bootstrap. The estimator supplies its own standard error and interval:

```
aipw = AIPTW(dfs, 'statin', 'Y')
aipw.treatment_model(g_model, g_estimator, bound=0.01)
aipw.outcome_model(q_model, q_estimator)
aipw.fit()
bias.append(aipw.risk_difference - truth)
stderr.append(aipw.risk_difference_se)
lcl.append(aipw.risk_difference_ci[0])
ucl.append(aipw.risk_difference_ci[1])
```

`sim_iptw.py` omits the `outcome_model` line (IPTW needs only the propensity score).

`sim_dcaipw.py` and `sim_dctmle.py` — the expensive ones

```
n_diff_splits = 100

for i in samples:
    dfs = df.loc[df['sim_id'] == i].copy()
    try:
        dcdr = DoubleCrossfitAIPTW(dfs, 'statin', 'Y')
        dcdr.treatment_model(g_model, g_estimator, bound=0.01)
        dcdr.outcome_model(q_model, q_estimator)
        dcdr.fit(resamples=n_diff_splits, method='median')
        bias.append(dcdr.risk_difference - truth)
        ...
    except:
        bias.append(np.nan)
        ...
```

The bare `except:` matters

A bare `except:` catches *everything*, records `np.nan`, and continues silently. If many datasets fail — say, because a propensity estimate hit zero — the run still completes and prints summary numbers computed from the survivors. **Always check your output CSV for NaN rows** before believing the numbers. `np.mean` would propagate NaN, so a visibly `nan` summary is the warning sign; but partial failures that still average are not visible without checking.

7 The output: what every number means

Each script builds a table with one row per simulated dataset and saves it as, e.g., `aipw_results1.csv` (2,000 rows plus a header).

7.1 The columns of the CSV

```
results['bias']   = bias                    # estimate - truth, per dataset
results['std']    = stderr                  # estimated standard error, per dataset
results['lcl']    = lcl                     # lower 95% confidence limit
results['ucl']    = ucl                     # upper 95% confidence limit
results['cover'] = np.where((results['lcl'] < truth) & (truth < results['ucl']),
    1, 0)
results['cld']    = results['ucl'] - results['lcl']
```

Column	Meaning
`bias`	$\hat{\psi}_k - \psi_{\text{true}}$ for dataset k. Note this is stored on the *bias* scale, whereas `lcl`/`ucl` are on the *estimate* scale — do not mix them up.
`std`	the standard error this method reported for dataset k.
`lcl`, `ucl`	the 95% confidence interval for dataset k.
`cover`	1 if that interval contained the truth, 0 otherwise.
`cld`	"confidence limit difference" — the width of the interval.

This CSV is the "full results" deliverable

Your supervisor's main scientific request is to present *everything in these files*, not the summary the authors chose to print. Each file holds 2,000 rows × 6 columns of information; the paper reports six numbers per estimator. Distributions, tails, failure rates and the relationship between estimated and actual error all live here and are absent from the publication. **Never delete these files.**

7.2 The six summary numbers printed to the console

```
print("Mean: ", np.round(np.mean(bias), decimal))
print("RMSE: ", np.round(np.sqrt(np.mean(results['bias'])**2 + np.std(bias, ddof
    =1)**2), decimal))
print("ASE:  ", np.round(np.mean(stderr), decimal))
print("ESE:  ", np.round(np.std(bias, ddof=1), decimal))
print("CLD:  ", np.round(np.mean(results['cld']), decimal))
print("Cover:", np.round(np.mean(results['cover']), decimal))
```

Mean — the bias

$$\text{Mean} = \frac{1}{2000}\sum_{k=1}^{2000}(\hat{\psi}_k - \psi_{\text{true}})$$

Should be near zero for a good estimator. **Near**, not exactly: with 2,000 datasets there is Monte Carlo error of roughly ESE/$\sqrt{2000}$, so deviations of a few thousandths are normal and should not be chased.

ESE — empirical standard error

$$\text{ESE} = \text{SD}\left(\hat{\psi}_1, \ldots, \hat{\psi}_{2000}\right)$$

How much the estimate *actually* bounces around across repeated studies. This is the truth about the estimator's precision, because we can see all 2,000 replications.

ASE — average standard error

$$\text{ASE} = \frac{1}{2000}\sum_{k=1}^{2000} \widehat{\text{SE}}_k$$

How much the method *claims* it bounces around, averaged over datasets.

ASE versus ESE — the single most informative comparison

This is exactly the check your supervisor described in his linear-regression walkthrough: compare the *mean of the estimated variances* against the *empirical variance of the estimates*.

- ASE $\approx$ ESE: the method's standard errors are honest. ✓
- ASE $<$ ESE: standard errors too small $\Rightarrow$ intervals too narrow $\Rightarrow$ coverage below 95%. This is the failure mode the paper is hunting for.
- ASE $>$ ESE: conservative — intervals wider than necessary. Expected for IPTW, by construction.

RMSE — root mean squared error

$$\text{RMSE} = \sqrt{\text{Mean}^2 + \text{ESE}^2}$$

Bias and variance combined into one number. Read the code carefully: `np.mean(results['bias'])**2` is *the mean of the bias, squared* — so this is $\sqrt{\text{bias}^2 + \text{variance}}$, the standard decomposition. A method can win on RMSE by being slightly biased but much more stable.

Cover — confidence interval coverage

$$\text{Cover} = \frac{1}{2000}\sum_{k=1}^{2000} \mathbb{1}[\text{lcl}_k < \psi_{\text{true}} < \text{ucl}_k]$$

The proportion of the 2,000 intervals that contained the truth. **Should be about 0.95.** This is the headline diagnostic: an estimator with good coverage gives you trustworthy inference; one with 0.80 coverage is lying to you one time in five.

Monte Carlo error on coverage is about $\sqrt{0.95 \times 0.05/2000} \approx 0.005$, so anything in roughly 0.94–0.96 is consistent with nominal.

CLD — confidence limit difference

$$\text{CLD} = \frac{1}{2000}\sum_{k=1}^{2000} (\text{ucl}_k - \text{lcl}_k)$$

Average interval width. Useful for comparing efficiency: among methods with correct coverage, narrower is better.

7.3 What you should see, and why

Setup	Estimator	Expected bias	Expected coverage
1	all six	≈ 0	≈ 0.95
2	g-comp, IPW	clearly non-zero	well below 0.95
2	AIPW, TMLE	non-zero (both models wrong)	below 0.95
3	g-computation	**noticeable bias**	**poor**
3	AIPW, TMLE	smaller bias	better
3	DC-AIPW, DC-TMLE	**smallest bias**	**closest to 0.95**

A subtlety about double robustness in setup 2

Double robustness protects you if *either* the propensity model or the outcome model is correct. In setup 2 *both* are misspecified, so AIPW and TMLE have no protection left and are also biased. This is not a bug — it is the correct behaviour, and a good thing to be able to explain.

8 `sim_single_example.py` — the smoke test

This script runs **every estimator on one dataset** and prints a result plus a timing for each. It is not part of the paper's results; it exists to check that the environment works.

```
bs_samples = 250          # bootstraps for g-computation
sample_splits_n = 100     # sample splits for the cross-fit procedures
...
bsl = superlearnersetup(var_type='binary', K=5)
```

Why this file is so useful for planning

Its repetition counts are **identical to production** (250 bootstraps, 100 splits) — only the number of cross-validation folds differs ($K = 5$ here versus $K = 10$ in the simulation scripts). That makes each of its eleven blocks a genuine measurement of the per-dataset cost of the corresponding production run. Multiply a block's elapsed time by 2,000 and you have a grounded runtime estimate for the whole study.

It prints eleven blocks — parametric and super-learner versions of g-formula, IPTW, AIPW, TMLE, DC-AIPW, DC-TMLE — each reporting `RD`, `SD(RD)`, the 95% limits, `CLD` and `TIME ELAPSED` (in minutes). Since the true effect is -0.108, every estimate should land somewhere in that neighbourhood; wild values indicate a broken environment.

9 Running the code today: three genuine incompatibilities

The code was written in 2019–2020. Three library idioms it uses have since been removed from the modern versions of those libraries. **The correct response is to run the code in an era-appropriate environment, not to edit the authors' files** — editing would compromise the reproduction.

9.1 scikit-learn: `penalty='none'`

```
LogisticRegression(penalty='none', solver='lbfgs', max_iter=1000)
```

Modern scikit-learn requires `penalty=None` (the object, not the string) and now deprecates the argument altogether. Appears in `super_learner.py` and every `sim_*.py`.

9.2 statsmodels: link classes versus link instances

```
f = sm.families.family.Binomial(sm.families.links.identity)
```

Here `identity` is passed as a *class*. Modern statsmodels requires an *instance*, `identity()`, and raises `TypeError: Calling Family(..) with a link class is not allowed`. This sits in `IPTW.fit()`, so it affects every IPTW run.

9.3 Python itself: `is` used for string comparison

```
if self.coef_method is 'SLSQP':
```

Identity comparison on a string literal. This happened to work on older CPython through string interning; modern Python emits a `SyntaxWarning` and the behaviour is not guaranteed. It is inside the super learner's weight optimiser — the code path used by every setup-3 run.

Why the pinned environment is the scientifically correct choice

Each of these could be "fixed" with a one-word edit. But the moment you edit the authors' files, you are no longer reproducing their code — you are reproducing your edited version of it, and any discrepancy becomes ambiguous. Building an environment with the library versions the authors used keeps their source byte-for-byte identical (verifiable with `git status`) and removes that ambiguity entirely. The incompatibilities are then findings to report, not problems to patch.

10 Glossary

Term	Meaning
ATE	Average treatment effect: the average difference in outcome if everyone were treated versus if nobody were. Here measured as a risk difference.
Risk difference	$\Pr(Y = 1 \mid \text{all treated}) - \Pr(Y = 1 \mid \text{none treated})$. The paper's effect scale; true value -0.1081508.
Confounder	A variable affecting both treatment and outcome; must be adjusted for.
Propensity score	$g(X) = \Pr(A = 1 \mid X)$, the probability of being treated given covariates.
Nuisance function	A quantity you must estimate on the way to the answer but do not care about (g and Q here).
Plug-in estimator	One that estimates a nuisance function and substitutes it directly — g-computation. Vulnerable to slow convergence.
Doubly robust	Consistent if *either* the propensity model or the outcome model is correct. AIPW and TMLE.
Influence function	The per-observation contribution to an estimator's error; its variance divided by n gives the estimator's variance.
Efficient influence function	The influence function achieving the smallest possible asymptotic variance. AIPW and TMLE are built from it.
Cross-fitting	Fitting nuisance models on one part of the data and evaluating the effect on another, so no observation informs its own prediction.
Double cross-fitting	Cross-fitting where the two nuisance functions are additionally estimated on *separate, disjoint* subsamples.
Super learner	A weighted combination of several candidate algorithms, with weights chosen by cross-validation. Also called stacking.
Monte Carlo error	The random noise from using 2,000 simulated datasets rather than infinitely many.
Coverage	The proportion of 95% confidence intervals that actually contain the truth. Should be ≈ 0.95.
ASE / ESE	Average estimated standard error / empirical standard error. Their agreement tests whether the reported uncertainty is honest.
Slow convergence ("slow bias")	The plug-in bias that arises because machine-learning nuisance estimates approach the truth more slowly than $1/\sqrt{n}$.

11 Quick reference

11.1 File map

File	Run it?	What it holds
`dgm.py`	once	data generation; n, `sims`, the seed, the truth
`super_learner.py`	never	`SuperLearner` class, the six-algorithm library
`estimators.py`	never	the six estimator classes and helpers
`sim_single_example.py`	smoke test	all estimators on one dataset, with timings
`sim_gform.py`	yes	g-computation (+ 250 bootstraps)
`sim_iptw.py`	yes	IPW
`sim_aipw.py`	yes	AIPW
`sim_tmle.py`	yes	TMLE
`sim_dcaipw.py`	yes	double cross-fit AIPW (100 splits)
`sim_dctmle.py`	yes	double cross-fit TMLE (100 splits)

11.2 Key constants

Constant	Value	Where
`n`	3000	`dgm.py`
`sims`	2000	`dgm.py`
random seed	1015033030	`dgm.py`
`truth`	−0.1081508	every `sim_*.py`
`bs_samples`	250	`sim_gform.py`
`n_diff_splits`	100	`sim_dcaipw.py`, `sim_dctmle.py`
`K` (super learner)	10	production setup 3
`K` (super learner)	5	`sim_single_example.py`
`bound`	0.01	every `treatment_model` call
number of splits	3	hard-coded in `_sample_split_`

11.3 Where each estimator's variance comes from

Estimator	Variance method	Cost
G-computation	external bootstrap (250 resamples)	251×
IPW	GEE robust sandwich (conservative)	1×
AIPW	influence function / n	1×
TMLE	efficient influence curve / n	1×
DC-AIPW	median of per-split variances + between-split term	600×
DC-TMLE	same	600×

11.4 Ten things most likely to trip you up

1. `IPTW.fit()` hard-codes the formula `'Y ~ statin'` and ignores the column names you passed in.
2. `GFormula` produces **no** standard error — hence the bootstrap, hence the runtime problem.
3. `TMLE` calls the treatment `self.exposure`, not `self.treatment`.
4. `TMLE`'s `alpha` argument is accepted and then ignored; z is fixed at 1.96 everywhere in the file.

5. `random_state` in the double cross-fit classes does nothing.
6. The number of cross-fit splits is **3**, hard-coded in five places.
7. Each nuisance model in the DC classes trains on only $n/3$ observations.
8. The DC standard error adds the between-split term *inside* the median, so it is not a simple sum of two variances.
9. The DC simulation scripts use a bare `except:` that silently records `np.nan` — check your CSVs for missing rows.
10. The outcome formula must contain the treatment term, or the estimated effect silently collapses.

11.5 Suggested reading order for the code

1. `dgm.py` in full — it is short and everything else refers to it.
2. `sim_gform.py` in full — the simplest complete script.
3. `GFormula` in `estimators.py` — about 110 lines.
4. `IPTW`, then `AIPTW`, then `TMLE`.
5. `superlearnersetup` and `SuperLearner.fit` in `super_learner.py`.
6. `_sample_split_` and `_single_crossfit_` — the rotation is the intellectual core of the paper.
7. `sim_dcaipw.py` to see it all assembled.

Every formula, constant and code excerpt in this document was read directly from the source files in `DoubleCrossFit/Python` and verified against them.

The authors' files are unmodified.